

EFREI

EYAGA NDEME SOLANGE DOMINIQUE

M1-CSM

Cours : Développement Sécurisé

Boris STOCKER

2026

1. Introduction

Ce rapport présente les outils et concepts abordés lors de nos séances de sécurisation avec Python. L'objectif est de consolider les connaissances acquises, notamment sur la validation de mots de passe, le hachage cryptographique, l'installation des bibliothèques nécessaires et l'utilisation d'un environnement de lab Linux.

Ces exercices pratiques ont été réalisés directement sur la plateforme de lab (direct.lab-boris.f), ce qui nous a permis de travailler dans un environnement réel et de tester nos scripts en conditions proches du terrain.

2. Environnement de travail

2.1 La plateforme de lab

Tous les exercices ont été réalisés sur un environnement Linux accessible via la plateforme direct.lab-boris.f. Chaque instance dispose d'une adresse IP propre (ex : 10.0.58.4) et est accessible en SSH :

```
ssh ip10-0-58-4-d85dkaadfpthb989ed60@direct.lab-boris.f
```

Cet environnement isolé permet de tester des scripts Python en sécurité, sans risque pour un réseau réel.

2.2 Installation des outils

Avant de commencer les exercices, il a fallu installer plusieurs paquets et bibliothèques. Voici les commandes utilisées dans le terminal Linux :

```
# Mise à jour du système et installation de nano
apk add nano

# Installation des bibliothèques Python nécessaires
pip install requests certifi charset-normalizer urllib3 idna
```

On peut voir dans le terminal les téléchargements effectués : urllib3, idna, charset-normalizer, certifi, requests. Ces bibliothèques sont nécessaires pour faire des requêtes HTTP depuis Python, par exemple pour vérifier si un mot de passe a été compromis en ligne.

2.3 Création et exécution d'un script

Une fois l'environnement prêt, on crée un fichier Python avec l'éditeur nano et on l'exécute :

```
# Création du fichier
nano script.py

# Exécution du script
python3 script.py
```

3. Validation de mot de passe

3.1 Principe

Un mot de passe solide est la première ligne de défense contre les accès non autorisés. La validation de mot de passe consiste à vérifier qu'un mot de passe saisi par l'utilisateur respecte un ensemble de critères de sécurité définis à l'avance.

Dans notre exercice, les critères imposés étaient les suivants :

- Au moins 12 caractères
- Au moins une lettre majuscule
- Au moins une lettre minuscule
- Au moins un chiffre
- Au moins un caractère spécial (!@#\$%^&*...)
- Aucun espace

3.2 Fonctionnement du script

Le script utilise le module `re` (expressions régulières) pour vérifier chaque critère. Si le mot de passe ne respecte pas un critère, un message d'erreur précis est affiché. Voici le code réalisé en cours :

```
importer la bibliotheque de hachage
importer re

def validate_password(mot_de_passe):
    erreurs = []
    si la longueur(mot_de_passe) < 8 :
        erreurs.append('Au moins 8 caracteres')
    si ce n'est pas re.search(r'[A-Z]', mot_de_passe):
        erreurs.append('Au moins une majuscule')
    si ce n'est pas re.search(r'[a-z]', mot_de_passe):
        erreurs.append('Au moins une minuscule')
    si ce n'est pas re.search(r'[0-9]', mot_de_passe):
        erreurs.append('Au moins un chiffre')
    si ce n'est pas re.search(r'[@#$$%^&*]', mot_de_passe):
        erreurs.append('Au moins un caractere special (!@#$$%^&*')')
    retourne erreurs
```

Et voici la version Python exacte :

```
import hashlib
import re

def validate_password(mot_de_passe):
    erreurs = []
    if len(mot_de_passe) < 8:
        erreurs.append('Au moins 8 caracteres')
    if not re.search(r'[A-Z]', mot_de_passe):
        erreurs.append('Au moins une majuscule')
    if not re.search(r'[a-z]', mot_de_passe):
        erreurs.append('Au moins une minuscule')
    if not re.search(r'[0-9]', mot_de_passe):
        erreurs.append('Au moins un chiffre')
    if not re.search(r'[@#$$%^&*]', mot_de_passe):
        erreurs.append('Au moins un caractere special (!@#$$%^&*')')
    return erreurs
```

3.3 Resultat observe

Lors de l'exécution, si on saisit un mot de passe faible comme '12345', le programme affiche les raisons du refus :

```
Mot de passe invalide :
```

- Au moins 8 caracteres
- Au moins une majuscule
- Au moins une minuscule
- Au moins un caractere special (!@#\$\$%^&*)

Si on saisit un mot de passe correct (ex : Bonjour!0), le programme repond :

Mot de passe accepte !

4. Hachage cryptographique avec hashlib

4.1 C'est quoi le hachage ?

Le hachage est une operation mathematique qui transforme une donnee (ici un mot de passe) en une empreinte numerique unique et de taille fixe, appelee hash ou digest. Cette operation est :

- Deterministe : le meme mot de passe donne toujours le meme hash
- Irreversible : on ne peut pas retrouver le mot de passe original depuis son hash
- Tres sensible : un seul caractere change produit un hash completement different

4.2 Pourquoi hacher les mots de passe ?

On ne stocke jamais un mot de passe en clair dans une base de donnees. Si la base est compromise, les mots de passe seraient directement exposes. En stockant seulement le hash, meme un attaquant qui vole la base ne peut pas retrouver les vrais mots de passe directement.

4.3 Implementation avec hashlib

Le module hashlib est integre a Python, aucune installation n'est necessaire. Voici la fonction utilisee en cours :

```
def hash_password(mot_de_passe):  
    mot_de_passe_octets = mot_de_passe.encode('utf-8')  
    objet_de_hachage = hashlib.sha256(mot_de_passe_octets)  
    renvoie_hash_objet.hexdigest()
```

Version Python exacte :

```
def hash_password(mot_de_passe):  
    mot_de_passe_octets = mot_de_passe.encode('utf-8')  
    objet_de_hachage = hashlib.sha256(mot_de_passe_octets)  
    return objet_de_hachage.hexdigest()
```

4.4 Resultat observe

Apres validation reussie du mot de passe, le programme affiche le hash SHA-256 correspondant. Par exemple pour le mot de passe 'Bonjour!0' :

Votre hash SHA-256 est :
d89819661cbf8424fd766d07aeee02b64b44ee50ec90ab2a7b770e66dab56b9d

Ce hash de 64 caracteres hexadecimaux (256 bits) est ce qui serait stocke en base de donnees a la place du vrai mot de passe.

4.5 Boucle de saisie

Le script tourne en boucle (tant que vrai) : il demande le mot de passe, vérifie les critères, et recommence jusqu'à ce que le mot de passe soit valide. Une fois valide, il demande une confirmation puis affiche le hash :

```
tant que vrai :  
    password = input('Entrez votre mot de passe : ')
```

5. Verification de mots de passe compromis

5.1 Principe

Dans un des exercices, on a vérifié si un mot de passe avait déjà été exposé dans une fuite de données. Le terminal affichait ce message :

```
Warning: password found 30272674 times in data breaches!
```

Cela signifie que le mot de passe testé (ici '1234') a été retrouvé plus de 30 millions de fois dans des bases de données de mots de passe volés. Il est donc complètement à éviter.

5.2 Comment ça fonctionne ?

Ce type de vérification utilise le service Have I Been Pwned (haveibeenpwned.com). Le principe repose sur le hachage partiel pour protéger la vie privée :

- On calcule le hash SHA-1 du mot de passe
- On envoie uniquement les 5 premiers caractères du hash à l'API
- L'API retourne tous les hashes commençant par ces 5 caractères
- On compare localement si notre hash complet est dans la liste
- Si oui : le mot de passe a été compromis, on affiche une alerte

Cette technique s'appelle k-anonymat : on ne transmet jamais le mot de passe ni son hash complet, ce qui protège la confidentialité de l'utilisateur.

6. Scapy — Scan du réseau et découverte des machines

6.1 Présentation de Scapy

Scapy est une bibliothèque Python puissante qui permet de créer, envoyer et analyser des paquets réseau. Dans nos exercices, on l'a utilisée directement depuis le terminal du lab pour scanner le réseau et identifier les machines actives.

On entre dans l'environnement Scapy avec la commande :

```
scapy
```

6.2 Scan ARP — Découverte des machines sur le réseau

Un scan ARP (Address Resolution Protocol) permet de découvrir toutes les machines actives sur un réseau local. On envoie un paquet Ethernet broadcast vers toutes les adresses et on observe qui répond. Voici le code utilisé en cours :

```
paquet_ethernet = Ether(dst='ff:ff:ff:ff:ff:ff') / paquet
resultat, _ = srp(paquet_ethernet, iface='eth0', timeout=2, verbose=1)
```

Explication des paramètres :

- Ether(dst='ff:ff:ff:ff:ff:ff') : envoie le paquet en broadcast (à toutes les machines du réseau)
- srp() : envoie et reçoit des paquets au niveau 2 (couche liaison)
- iface='eth0' : utilise l'interface réseau eth0
- timeout=2 : attend 2 secondes les réponses

6.3 Resultat observe

Le terminal affiche :

```
Finished sending 256 packets.
Received 5 packets, got 5 answers, remaining 251 packets
```

Puis après affichage des résultats, on a obtenu la liste des IP et adresses MAC actives sur le réseau :

```
Adresses IP actives sur le reseau :
IP: 10.0.121.1   MAC: 06:20:fa:13:d3:12
IP: 10.0.121.3   MAC: 02:42:0a:00:79:03
IP: 10.0.121.4   MAC: 02:42:0a:00:79:04
IP: 10.0.121.5   MAC: 02:42:0a:00:79:05
IP: 10.0.121.6   MAC: 02:42:0a:00:79:06
Scan termine.
```

On retrouve bien les 4 instances du lab (node1 à node4) avec leurs adresses MAC respectives. Chaque machine répond au broadcast ARP, ce qui confirme qu'elle est active sur le réseau.

6.4 Code d'affichage des résultats

```
print('Adresses IP actives sur le reseau :')
for _, recu in resultat:
    print('IP:', recu.psrc, '| MAC:', recu.hwsrc)
print('Scan termine.')
```

7. Capture de trafic réseau avec tcpdump ---

7.1 Installation de tcpdump

tcpdump est un outil en ligne de commande qui permet de capturer et analyser le trafic réseau en temps réel. Il s'installe sur le lab avec :

```
apk add tcpdump
```

Le terminal a confirmé l'installation : tcpdump 4.99.4-r1, 479 MiB utilisés.

7.2 Capture du trafic sur le port 80

Depuis l'environnement Scapy du lab, on a capture le trafic TCP sur l'interface eth0, filtre sur le port 80 (HTTP) :

```
tcpdump -i eth0 tcp and port 80
```

Explication de la commande :

- -i eth0 : écoute sur l'interface réseau eth0
- tcp : filtre uniquement le protocole TCP
- and port 80 : uniquement sur le port 80 (trafic HTTP)

7.3 Resultat observe

Le terminal a affiche en temps reel les paquets TCP captures, avec pour chaque paquet : l'horodatage, l'IP source, l'IP destination, les flags TCP et la taille :

```
08:54:24.876815 IP 142.246.179.29 > node1.80: Flags [S], seq 0, win 8192, length 0
08:54:24.909997 IP 1-36-220-230.ptr.netvigator.com > node1.80: Flags [S], seq 0
08:54:24.958134 IP 18.9.158.66.20 > node1.80: Flags [S], seq 0, win 8192, length 0
```

Le flag [S] signifie SYN : ce sont des tentatives de connexion TCP entrantes vers le port 80 de notre machine. On observe des connexions venant du monde entier, ce qui montre que notre machine du lab est exposee sur internet.

7.4 Le sniffer Scapy

Scapy dispose egalement d'un sniffer integre. Le compteur affiche en temps reel les paquets captures par type de protocole :

```
^CSniffed: TCP:0 UDP:0 ICMP:0 Other:0>
>> exit()
```

Ctrl+C arrete la capture. On sort ensuite de Scapy avec exit().

8. Recapitulatif des outils utilises

Outil / Module	Role	Commande d'installation
hashlib	Hachage SHA-256 des mots de passe	Integre a Python (aucune)
re	Validation par expressions regulieres	Integre a Python (aucune)
requests	Requetes HTTP vers l'API externe	pip install requests
Scapy	Scan reseau ARP, sniffing de paquets	scapy (deja installe)
tcpdump	Capture de trafic reseau en temps reel	apk add tcpdump
nano	Editeur de texte en ligne de commande	apk add nano
python3	Execution des scripts	Integre a Linux

9. Conclusion

Ces seances nous ont permis de decouvrir des aspects concrets et pratiques de la securisation en Python. On a appris que la securite commence par des bases solides : imposer des criteres stricts sur les mots de passe, ne jamais les stocker en clair, et utiliser des algorithmes de hachage reconnus comme SHA-256.

L'utilisation d'un environnement de lab reel (direct.lab-boris.f) a rendu l'apprentissage plus concret : on a installe des bibliotheques, cree des scripts, teste des cas valides et invalides, et interprete les resultats.

Ces connaissances serviront de base pour aller plus loin en securite informatique : analyse reseau, detection d'intrusions, chiffrement de fichiers, etc.